Reversing by Doing

Free sample — Introduction & Chapter 4 · the full book: payhip.com/ReversingbyDoing

---

Introduction

Why this book exists

Most reverse engineering material fails in one of two ways. Academic textbooks spend 200 pages on instruction encodings before you touch a binary. Tutorial videos show a wizard flying through a debugger at 2× speed, and you finish knowing that wizards exist.

Neither works, because reversing isn't knowledge — it's a **skill**. You build it the way you build any skill: reps, failure, and slow accumulation of pattern recognition.

So this book is structured around one rule: **you break something in every chapter.** The theory arrives in the exact moment you need it, at the exact size you need it, and never a page earlier. By the end you will have patched password checks, written a keygen, unpacked a binary by hand, beaten anti-debug three different ways, hooked a live Android app, and shipped your own automated triage tool in Python. Those aren't metaphors. They're the fifteen labs this book is built on.

Who this is for

- You can read basic C (functions, loops, pointers without panicking). That's the only prerequisite.
- You don't know assembly — Chapter 2 gives you a survival guide, not a reference tomb.
- You want a security career, you're curious how software really works, or you just like opening things. All three fit.

Who this is not for

- People wanting to pirate commercial software. Every target in this book was written for this book (or is software you own). The skills transfer to legitimate work — malware analysis, vulnerability research, CTFs, compatibility engineering. The book never targets anyone's product.
- People wanting to skip to "the exploits." This is the foundation layer. It's the boring-done-properly part that makes everything later possible.

How to use this book

Each lab chapter follows the same rhythm:

- **Meet the target** — what it does when you run it.
- **Recon** — what you can know before executing anything.

- **The attempt** — you try first. Seriously. Set a 30-minute timer.
- **The walkthrough** — fully annotated, every click explained, including the wrong turns.
- **Exercises** — variations that force you to generalize, not copy-paste.

The labs binaries ship with the book (Ebook + Labs tier): every binary plus its full source code, so when you're stuck you can *peek at the source after you've earned it*. The walkthroughs grade you honestly — a solution you don't understand is a solution you don't have.

The legality box

Reversing is legal in most jurisdictions when the software is yours, when it's expressly permitted, or when it's material written for practice. Everything in this book fits:

- All lab binaries were written for this book. Break them guilt-free.
- Appendix C lists legal practice grounds: crackmes.one (targets explicitly made to be broken), CTF challenges, and your own software.
- Nothing in this book provides instructions for defeating licensing or protection on third-party commercial products. That's not a legal disclaimer we hide behind — it's the design of the book.

If your interest is professional — incident response, malware analysis, security research — you're in the right place: Chapters 10, 11, and 15 are the daily workflow of a malware analyst, compressed.

The toolchain costs nothing

Ghidra (NSA, free), x64dbg (open source), Detect It Easy, Frida, jadx, Python with Capstone and pefile. Every tool in this book is free and standard — the same names professionals open first. Total software budget for this course: **$0**.

One promise

You will get stuck. The walkthrough will be right there, but use it the way a climbing partner spots you — not the way an elevator does. The stuck-ness is where the skill actually forms.

Let's break things.

— *[AUTHOR NAME]*

Chapter 4 — First Blood: Finding the Password Check

> t; **In this chapter you will:** take a program that asks for a password, find the exact instruction that decides your fate, and change it so it accepts anything you type. Recon → strings → cross-references → decompilation → one clean patch. This is the entire reversing loop in miniature — everything later in the book is this loop with harder targets.

Tools: Ghidra, x64dbg, Detect It Easy, HxD (or any hex editor)

Target: `lab04.exe` (Windows x64) — full source included below so you can verify every claim

Time: 60–90 minutes

4.1 Meet the target

Run `lab04.exe`. It's underwhelming, deliberately:

```
lab04 — chapter 4 practice target
Enter password: hello
Wrong password.
```

It asks for a password, compares it to something, and prints a verdict. There is no hint, no error code, no network call. Your goal for this chapter, stated precisely:

> t; **Make the program print `Access granted.` for any input — and understand exactly why your patch works.**

Here is the entire source of the target. In the real world you don't get this — but while learning, checking your conclusions against ground truth is how pattern recognition actually forms:

```
// lab04.c — build: x86_64-w64-mingw32-gcc lab04.c -o lab04.exe -s
#include <stdio.h>
#include <string.h>

int main(void) {
    char buf[64];

    printf("lab04 — chapter 4 practice target\n");
    printf("Enter password: ");
    if (scanf("%63s", buf) != 1)
        return 1;

    if (strcmp(buf, "hunter2") == 0)
        printf("Access granted.\n");
    else
        printf("Wrong password.\n");

    return 0;
}
```

4.2 Recon: never execute first

Before running anything (you already ran it — forgive yourself, then stop doing that), you profile the binary statically. This habit will matter enormously when targets stop being friendly.

Detect It Easy. Drop `lab04.exe` into DIE:

- **Compiler:** MinGW GCC 13.x — tells you the calling convention is Microsoft x64 and the runtime is msvcrt.

- **Packer:** none. Plain PE64, 3 sections (`.text` , `.rdata` , `.data`). If this said UPX or "borland/lzma", Chapter 10 territory — you'd unpack before anything else.
- **Entry point:** in CRT startup code, not `main` . Normal. The real entry work happens later.

Hash it. `Get-FileHash lab04.exe` (or `sha256sum` on Linux). When you publish a walkthrough or compare notes with anyone, you refer to the binary by hash. This is professional hygiene, start now:

```
SHA-256: 1f0e...(fill when you build your labs)
```

4.3 Strings: the first cheat code

Every string literal in the binary sits in `.rdata` , readable as plain bytes. Fire Ghidra: new project → import → CodeBrowser → Analysis (auto, default options are fine for now) → **Search → For Strings**.

Filter by "password". You should find, among the noise:

```
Access granted.
Wrong password.
Enter password:
```

Double-click `Access granted.` — Ghidra jumps to its address in `.rdata` , e.g. `0x140002340` . The string is just bytes; the *code that uses it* is what you want.

4.4 Cross-references: from data to the code that cares

With your cursor on the string, press **Ctrl+Shift+F** (References → Show References to Address). One result: an `lea` instruction in a function around `0x140001060` .

That `lea` is the compiler saying "load the *address* of this string, I'm about to pass it to something." Click through. Ghidra shows you the disassembly; press **F5**... no wait — in Ghidra the decompiler view is already open on the right, all the time. (That F5 reflex is from IDA. Unlearn it.)

Here's roughly what you'll see in the decompiler, with Ghidra's default names:

```
undefined8 FUN_140001060(void)
{
    char acStack_48 [64];
    int iVar1;
    undefined4 extra_var;

    printf("lab04 — chapter 4 practice target\n");
    printf("Enter password: ");
    __isoc99_scanf(&DAT_14000230c,acStack_48);
    iVar1 = strcmp(acStack_48,&DAT_140002318);
    if (iVar1 == 0) {
        puts("Access granted.");
    }
    else {
        puts("Wrong password.");
    }
```

```
    return 0;
}
```

Read that again. Ghidra just handed you the whole program. Ten seconds ago this was an opaque binary; now it's an `if/else`. Note what the decompiler did for you:

- `DAT_140002318` — that's `"hunter2"` sitting in `.rdata`. Click it to confirm.
- `acStack_48` — your input buffer on the stack.
- The entire "protection" is one comparison against zero.

Do the rename ritual now (it becomes reflex): right-click `FUN_140001060` → Edit Function Signature → rename to `main`. Rename `acStack_48` → `user_input`. The decompiler view updates live; future-you reading this function in a week will be grateful.

4.5 The moment: the conditional jump

Switch from the decompiler to the Listing (disassembly) — they're synchronized, same function. Around the `strcmp` call you'll see this shape:

```
lea     rdx,[rbp-0x48]              ; arg2: your input (user_input)
lea     rcx,[rip+0x...]             ; arg1: "hunter2"
call    strcmp
test    eax,eax                     ; strcmp == 0 means MATCH
jne     WRONG_LABEL                 ; not equal → jump to failure
lea     rcx,[rip+0x...]             ; "Access granted."
call    puts
jmp     DONE
WRONG_LABEL:
lea     rcx,[rip+0x...]             ; "Wrong password."
call    puts
DONE:
; ... epilogue
```

(MinGW build via msvcrt — if your toolchain differs, register names or thunks may vary, but the *shape* — `call strcmp / test eax,eax / jcc` — is universal. This exact triplet is one of those ~20 patterns from Chapter 2.)

Understand each piece, because you're about to edit it:

- `strcmp` returns `0` when strings are **equal** (not 1 — everyone gets this wrong the first week).
- `test eax,eax` sets ZF=1 exactly when EAX==0, i.e. when the password matched.
- `jne` ("jump if not zero") fires when ZF==0 — when the password is wrong — sending you to the failure branch.
- **The entire security decision is one byte of opcode away from being yours.** `jne` short is `75 xx`. `je` short is `74 xx`. One bit.

4.6 The plan: three ways to win

There are always multiple patches. Ranking them sharpens your judgment:

- **Patch the jump** (best): `jne` → `nop nop` (remove the failure exit) or `jne` → `je` (invert — then the *wrong* password wins, which is funnier and slightly more fragile).
- **Patch the comparison result**: breakpoint after `strcmp`, zero EAX in the debugger. Powerful, but it's a *runtime* patch — you must re-do it every run. Cheating for testing, not a solution.
- **Patch the string**: overwrite `"hunter2"` in `.rdata` with your own password (same or shorter length, null-terminated). Now the binary genuinely checks *your* password. Least invasive, most "legitimate" feeling — and in real reversing, string patches are often the cleanest fix for config values, not just passwords.

You'll do all three. All of them teach a different tool move.

4.7 Dynamic: doing it in x64dbg

Close Ghidra (leave it open actually — real reversers run both side by side; static for the map, dynamic for the truth). Open `lab04.exe` in **x64dbg**.

Set the breakpoint on strcmp. In the command box at the bottom:

```
bp strcmp
```

Run (F9). The program starts, asks for input — type anything, say `letmein`, Enter. Boom: breakpoint hits inside msvcrt's `strcmp` implementation. You don't care about its internals; you want the moment it *returns*. Press F8 (step over) — hmm, you're *inside* strcmp, so F8 steps through its guts. Two options, learn both:

- Press **Ctrl+F9** ("run to return") — executes until strcmp returns — then a single **F8** to land back in `main`, right after the `call`.
- Or, back up: before running, set a hardware/software breakpoint on the `test eax,eax` instruction in `main` instead. Both patterns are standard; "break on the interesting *caller* instruction" is usually faster than breaking in the library.

You're now sitting *after* the call, *before* the verdict. Look at the **Registers** panel (top right): `EAX` holds strcmp's return — nonzero (your `letmein` ≠ `hunter2`).

Steal the decision, option 2 from the plan: right-click EAX in the registers panel → modify → set to `0` → F9.

```
Access granted.
```

You just committed your first runtime patch. The program believed a lie you told its registers. Savor it briefly — then note the limitation: run the binary again and it's stubborn again. Runtime patches prove understanding; they don't change the program.

4.8 The permanent patch

Back to the `jne` instruction. In x64dbg:

- Break on it (double-click its address / F2 on the line).
- Run, type wrong password, land on the `jne`.

- Right-click → **Assemble** (or press Space). Type `nop`. The assembler replaces the 2-byte `jne rel8` — wait, it replaces *one* byte. You have `75 xx` (2 bytes) becoming `90` (1 byte), leaving `xx` as a stray opcode. **This is the classic beginner corruption** and exactly why you check sizes: right-click → **Binary → Fill with NOPs** on the *second* byte too, or assemble `nop` twice. When you're done, the two bytes at that address are `90 90` and the disassembly shows two clean `nop`s.
- F9.

```
Access granted.     ← for literally any input
```

Make it permanent: the patch currently lives only in this debug session's memory. In x64dbg: right-click the patched region → **Patch file** (Ctrl+P) → review the diff (`75 xx → 90 90`) → **Patch file**, save as `lab04_patched.exe`.

Verify outside the debugger:

```
.\lab04_patched.exe
Enter password: garbage
Access granted.
```

Exercise 4.1 — the invert. Instead of NOP-ing, patch `75` (jne) to `74` (je). Predict the behavior *before* running, then run. Explain what happens with the correct password. This one-line exercise kills more misconceptions than a page of text.

Exercise 4.2 — the string patch. In HxD, open the original `lab04.exe`, Search → `hunter2` (ASCII). Overwrite with your own password of length ≤ 7, pad the remainder with `00`. Save, run, use your password. Why must the replacement be *shorter or equal*? What breaks if it's longer? (Hint: what sits after `"hunter2"` in `.rdata`?)

Exercise 4.3 — the bug hunt. Change the `nop nop` patch so that the program prints *both* messages for one run. (You'll have to think about where the `jmp DONE` is and why it exists.) This is your first taste of Chapter 5: patches have consequences.

4.9 What you actually learned

The loop — recon → strings → xrefs → decompile → locate decision → patch → verify — is not a chapter trick. It is *the* reversing loop. Everything ahead is the same loop with escalating resistance:

- Targets without helpful strings (Ch 6)
- Targets that detect your debugger (Ch 9)
- Targets that aren't really "there" until unpacked (Ch 10)
- Targets whose logic lives in Java/managed code (Ch 13–14)

And you learned the meta-skill: **the binary is not a black box; it's a very honest machine that always does exactly what its bytes say.** From here on, you're just learning to read faster.

> t; **Lab 04 (graded):** make `lab04.exe` accept any password using *each* of the three methods — jump patch (file-level), register patch (runtime), string patch (file-level). Screenshot all three. Then do Exercise 4.1 and 4.2. When you can explain *to a rubber duck* why `test eax,eax / jne` encodes "wrong → fail", you're ready for Chapter 5.

*Next chapter: **Patching — NOP, Invert, Redirect.** Now that you've won once, we make you lose on purpose: state that corrupts, jumps that can't be inverted, and the three questions you ask before touching any byte.*